NANYANG TECHNOLOGICAL UNIVERSITY

EE7207 Neural Networks & Deep Learning

Assignment 2 Report

MSc in Electrical and Electronic Engineering
Yan Ziming
G2507084J

April 2, 2026

Contents

1 Introduction **1**
1.1 Problem Statement 1
1.2 Data Sourcing & Preparation 1

2 Methodology **1**
2.1 Model Choice: BERT-base-uncased 1
2.2 Pre-trained Foundation and Architecture 2
2.3 Baseline: Vanilla BERT Classifier 2
2.3.1 Architecture Modification 2
2.3.2 Baseline Training Strategy 2
2.4 Fine-Tuning 2
2.4.1 LoRA (PEFT) 3
2.4.2 Adaptation Strategy 3
2.4.3 Implementation 3
2.5 Hyperparameters 3

3 Results **4**
3.1 Performance Metrics 4
3.2 Analysis 5

4 Discussion **5**
4.1 Obstacles & Solutions 5
4.2 Future Directions 6

5 Appendix **7**
5.1 data_loader.py 7
5.2 model_lib.py 8
5.3 train.py 9
5.4 inference.py 10

1 Introduction

1.1 Problem Statement

In this project, I address the task of **Financial Sentiment Classification**. Financial texts often contain subtle linguistic cues where general-purpose models fail to capture the domain-specific context (e.g., ”growth below expectation” is negative despite the word ”growth”). The objective is to classify financial news sentences into three polarities: *Positive*, *Negative*, and *Neutral*.

1.2 Data Sourcing & Preparation

- **Source:** Financial PhraseBank (Sentences with 50% agreement).
- **Size:** Approximately 4,846 sentences.
- **Preprocessing:**
  1. Tokenization using the BERT WordPiece tokenizer.
  2. Sequence padding and truncation to a maximum length of 128 tokens.
  3. Label encoding for the three sentiment classes.

A critical challenge in the Financial PhraseBank dataset is **class imbalance**. As shown in the data analysis, the *Neutral* class significantly outweighs the *Positive* and *Negative* classes. To ensure the model generalizes well across all sentiments, we adopted a stratified split to maintain the original distribution:

- **Training Set (80%):** Used for weight updates and adapter tuning.
- **Validation Set (10%):** Used for hyperparameter tuning and early stopping.
- **Test Set (10%):** Reserved for final unbiased evaluation.

This rigorous partitioning ensures that the performance metrics reported are not inflated by the majority class.

2 Methodology

2.1 Model Choice: BERT-base-uncased

I selected **BERT (Bidirectional Encoder Representations from Transformers)** over traditional RNNs like LSTM. Unlike LSTMs which process text sequentially, BERT’s self-attention mechanism allows it to capture global dependencies and bi-directional context simultaneously. This is particularly crucial for financial nuances where the meaning of a term often depends on both preceding and succeeding words.

2.2 Pre-trained Foundation and Architecture

The core of system is the `bert-base-uncased` model, sourced from the **Hugging Face Model Hub**. This model was pre-trained on a massive corpus (Wikipedia and BookCorpus) using two self-supervised tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

- **Architecture:** 12 Transformer blocks, 768-dimensional hidden states, and 12 attention heads.
- **Parameters:** Approximately 110 million total parameters.

2.3 Baseline: Vanilla BERT Classifier

To establish a rigorous performance ceiling for comparison, I first implemented a **Vanilla BERT Classifier** as baseline.

2.3.1 Architecture Modification

The standard BERT model outputs a hidden state for each input token. For the classification task, we utilize the final hidden state of the **[CLS]** token, which serves as a dense representation of the entire input sequence. We then append a task-specific **Multi-Layer Perceptron (MLP)** head:

$$P = \text{Softmax}(W \cdot h_{[CLS]} + b) \tag{1}$$

where $W \in \mathbb{R}^{K \times 768}$ is the weight matrix and $K = 3$ is the number of sentiment labels.

2.3.2 Baseline Training Strategy

In the baseline configuration, the pre-trained BERT backbone is **frozen** (weights are not updated). Only the parameters of the newly added MLP classifier are trained with learning rate (lr) = 2×10^{-3}. This allows us to measure the raw feature-extraction capability of the general-purpose BERT model when applied to the financial domain without specific task adaptation.

2.4 Fine-Tuning

The baseline is a general human language classifier, which only knows the rules and format of human language, but may not adequately capture domain-specific nuances. In order to adapt the classifier from a general-purpose model to a specialized financial model, Supervised Fine-Tuning (SFT) is applied.

2.4.1 LoRA (PEFT)

As I mentioned before, there are about 110M parameters for **BERT** model, if I train all the weight parameters, the computational cost is very high. To overcome the computational cost of full fine-tuning, we employ **Low-Rank Adaptation (LoRA)**.

$$h = W_0 x + \Delta W x = W_0 x + BAx \tag{2}$$

where $B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$ are trainable low-rank matrices ($r = 8$).

2.4.2 Adaptation Strategy

Unlike the Baseline (Linear Probing), which only trains the top-level MLP head, our LoRA approach injects trainable adapters directly into the Transformer backbone. Specifically, we target the `query` and `value` projection matrices within the multi-head self-attention layers. By adapting these internal attention mechanisms, the model can "re-learn" how to attend to domain-specific financial terminology (e.g., distinguishing between a "market correction" and a "grammatical correction") without altering its fundamental linguistic capabilities.

2.4.3 Implementation

The LoRA integration was implemented using the `peft` library. Below is the core configuration used to inject trainable matrices into the BERT attention layers:

Listing 1: LoRA Configuration Snippet

```
from peft import LoraConfig, get_peft_model
peft_config = LoraConfig(
            task_type=TaskType.SEQ_CLS,
            r=r,
            lora_alpha=lora_alpha,
            target_modules=["query", "value"],
            lora_dropout=dropout,
            modules_to_save=["classifier"]
        )
lora_model = get_peft_model(model, peft_config)
```

2.5 Hyperparameters

In our experiments, we set the number of training epochs to 3. This decision is primarily driven by the architectural characteristics of LoRA (Low-Rank Adaptation). Compared to full-parameter fine-tuning, LoRA significantly reduces the number of trainable parameters (in our case, only 297219 parameters need to be updated, which is 0.27% of the total parameters). Small epoch could avoid the risk of overfitting efficiently.

Hyperparameter	Value
Learning Rate	2×10^{-4}
Batch Size	32
Epochs	3
LoRA Rank (r)	8
LoRA Alpha	16
Optimizer	AdamW

Table 1: Fine-tuning Hyperparameters

3 Results

3.1 Performance Metrics

We compared the LoRA-enhanced model with Vanilla BERT Classifier model by metrics, like accuracy, precision, and F1 score as shown in following table. In addition, for the three-classes classification problem, the confusion matrix provides a granular view of the model's predictive capability. In the visualized matrix, the diagonal elements represent correct classifications (True Positives), while the off-diagonal elements indicate specific sentiment misalignments.

Method	Accuracy	Precision	Recall	F1-Score
Linear Probing (Baseline)	0.52	0.45	0.52	0.48
LoRA Fine-tuning	**0.83**	**0.81**	**0.83**	**0.80**

Table 2: Model Evaluation Results

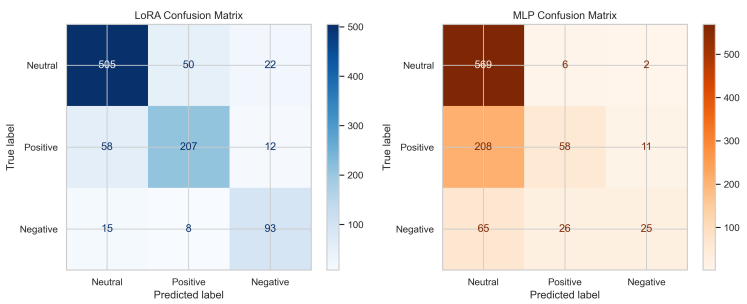

Figure 1: Confusion Matrices

It is obvious that the fine tuning optimize the performance of model significantly, with little computation resource consumed.

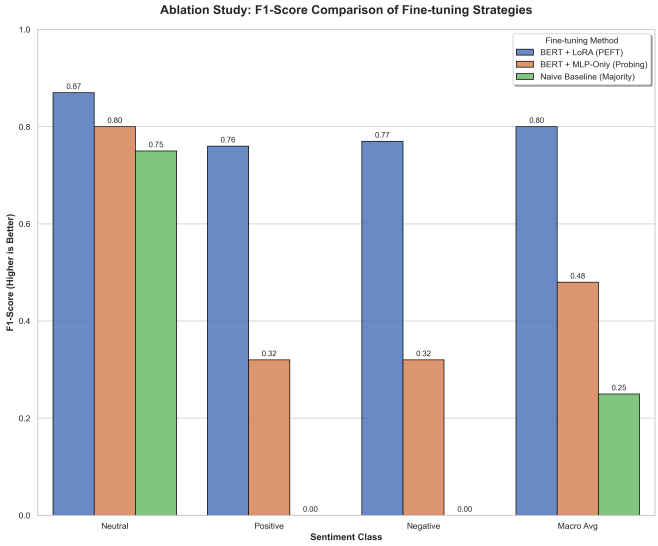

Figure 2: Comparison Between Models

3.2 Analysis

The substantial performance gap between **Linear Probing** (F1: 0.48) and **LoRA Fine-tuning** (F1: 0.80) highlights the limitations of static feature extraction.

In the baseline scenario, the frozen BERT encoder treats financial terms with general linguistic semantics. For instance, the word *"decrease"* might be encoded as a neutral movement in general text, but in a financial context like *"decrease in net profit"*, it carries a strong negative polarity. By adapting the `query` and `value` projection matrices via LoRA, the model learns to **re-weight its attention**, effectively "fine-tuning" its focus toward domain-specific keywords that were previously overlooked by the general-purpose backbone.

4 Discussion

4.1 Obstacles & Solutions

One major obstacle encountered during the initial training phase was **Loss Stagnation**. Despite the theoretical efficiency of LoRA, our early experiments yielded an F1-score of approximately **0.5**, suggesting the model was performing no better than random guessing. Upon analyzing the training logs, we observed that the loss remained trapped around 1.0 with negligible improvement over the first half-epoch:

$$\{\text{'loss': '1.094', 'epoch': '0.08'}\} \rightarrow \{\text{'loss': '1.017', 'epoch': '0.24'}\} \rightarrow \{\text{'loss': '0.970', 'epoch': '0.45'}\}$$

The stagnation in loss, coupled with the limited update steps (only 3 epochs), indicated that the initial learning rates (LoRA: 5×10^{-5}, MLP: 3×10^{-4}) were too conser-

vative. The weight updates were insufficient to move the low-rank adapters from their initialization toward a global minimum within the short training window.

To overcome this, we recalibrated the optimization strategy by increasing the learning rates to 2×10^{-4} **for LoRA** and 2×10^{-3} **for the MLP head**. This more aggressive approach provided the necessary ”gradient push” for the small set of trainable parameters. Following this adjustment, the loss showed a consistent downward trend, and the F1-score successfully converged to its peak value, validating that parameter-efficient methods often require higher learning rates than traditional fine-tuning.

4.2 Future Directions

Implementing Retrieval-Augmented Generation to provide the model with real-time market context (e.g., 2026 Q1 GDP data) to further reduce hallucinations in sentiment boundary cases.

5 Appendix

5.1 data_loader.py

```python
import torch
import os
from transformers import AutoTokenizer
from datasets import Dataset

class LocalFinancialDataLoader:
    def __init__(self, model_name: str, max_length: int = 128):
        """
        :param model_name
        :param max_length
        """
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.max_length = max_length

        self.label_map =  "neutral": 0, "positive": 1, "negative": 2}
                          {

    def _load_and_parse(self, file_path: str):
        data =  "sentence": [], "labels": []}
                {
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"未找到文件: {file_path}")

        with open(file_path, 'r', encoding='latin-1') as f:
            for line in f:
                line = line.strip()
                if "@" in line:
                    parts = line.split("@")
                    sentence = "@".join(parts[:-1])
                    sentiment = parts[-1].lower()

                    if sentiment in self.label_map:
                        data["sentence"].append(sentence)
                        data["labels"].append(self.label_map[sentiment])

        return Dataset.from_dict(data)

    def _tokenize_fn(self, examples):
        return self.tokenizer(
            examples["sentence"],
            padding="max_length",
            truncation=True,
            max_length=self.max_length
        )

    def prepare_datasets(self, file_name: str, data_dir: str = "Assignments/A2/data", test_size=0.2, seed=42):
        """
        核心流程: 读取 -> 转换 -> Tokenize -> 切分
        """
        file_path = os.path.join(data_dir, file_name)

        # 1. 加载并解析本地文本
        raw_dataset = self._load_and_parse(file_path)

        # 2. 特征编码 (Feature Encoding)
        tokenized_ds = raw_dataset.map(
            self._tokenize_fn,
            batched=True,
            remove_columns=["sentence"]
        )

        tokenized_ds.set_format("torch")
        split_ds = tokenized_ds.train_test_split(test_size=test_size, seed=seed)

        return split_ds["train"], split_ds["test"]
```

5.2 model_lib.py

```python
import torch
from transformers import AutoModelForSequenceClassification
from peft import LoraConfig, get_peft_model, TaskTyp
                e
class FinancialModelBuilder:
    def __init__(self, model_name: str = "bert-base-uncased", num_labels: int = 3):
        self.model_name = model_name
        self.num_labels = num_labels

    def build_lora_model(self, r: int = 8, lora_alpha: int = 16, dropout: float = 0.1):
        """
        机理: LoRA 模式 - 在 Transformer 内部注入低秩矩阵 (BA) 并训练分类头
        """
        model = AutoModelForSequenceClassification.from_pretrained(
            self.model_name,
            num_labels=self.num_labels
        )

        peft_config = LoraConfig
            task_type€TaskType.SEQ_CLS,
            r=r,
            lora_alpha=lora_alpha,
            target_modules=["query", "value"],
            lora_dropout=dropout,
            modules_to_save=["classifier"]
        )

        lora_model = get_peft_model(model, peft_config)

        print("--- [MODE: LoRA] Trainable Parameters Status ---")
        lora_model.print_trainable_parameters()

        return lora_model

    def build_mlp_only_model(self):
        """
        机理: Linear Probing 模式 - 彻底冻结 BERT Backbone, 仅开放顶层分类头 (MLP)
        """
        # 1. 加载原始序列分类模型
        model = AutoModelForSequenceClassification.from_pretrained(
            self.model_name,
            num_labels=self.num_labels
        )

        # 2. 遍历所有参数, 手动执行冻结逻辑
        for name, param in model.named_parameters():
            # 只有名字里包含 'classifier' 的参数 (即最顶层的 MLP) 才允许计算梯度
            if "classifier" in name:
                param.requires_grad = True
            else:
                param.requires_grad = False

        # 3. 打印参数状态用于 A2 报告对比
        trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
        all_params = sum(p.numel() for p in model.parameters())
        print("--- [MODE: MLP Only] Trainable Parameters Status ---")
        print(f"trainable params: {trainable_params} || all params: {all_params} || trainable%: {100 *
trainable_params / all_params:.4f}")

        return model

# --- 测试逻辑 ---
if __name__ == "__main__":
    builder = FinancialModelBuilder()

    print("\n[Testing LoRA Construction...]")
    m_lora = builder.build_lora_model(r=8)

    print("\n[Testing MLP-Only Construction...]")
    m_mlp = builder.build_mlp_only_model()
```

5.3 train.py

```python
import os
import argparse
import numpy as np
import torch
import evaluate
from transformers import TrainingArguments, Trainer
from config import Config
from data_loader import LocalFinancialDataLoader
from model_lib import FinancialModelBuilder

def parse_args():
    parser = argparse.ArgumentParser(description="EE7207 A2 Training Script")
    # 核心：增加模式切换参数
    parser.add_argument('--mode', type=str, default='lora', choices=['lora', 'mlp'],
                        help="训练模式：'lora' (PEFT 适配) 或 'mlp' (仅训练输出层)")
    parser.add_argument('--r', type=int, default=Config.LORA_R, help="LoRA 的秩 (Rank)")
    parser.add_argument('--batch', type=int, default=Config.BATCH_SIZE)
    parser.add_argument('--epochs', type=int, default=Config.EPOCHS)
    parser.add_argument('--lr', type=float, default=Config.LEARNING_RATE)
    return parser.parse_args()

def run_experiment():
    args = parse_args()

    # 1. 环境与数据准备
    loader = LocalFinancialDataLoader(Config.BASE_MODEL_NAME, Config.MAX_LENGTH)
    train_ds, test_ds = loader.prepare_datasets(
        file_name="Sentences_50Agree.txt",
        seed=Config.SEED
    )

    # 2. 动态构建模型
    builder = FinancialModelBuilder(Config.BASE_MODEL_NAME)

    if args.mode == 'lora':
        model = builder.build_lora_model(r=args.r, lora_alpha=Config.LORA_ALPHA)
        experiment_id = f"lora_r{args.r}_b{args.batch}_e{args.epochs}"
    else:
        # 消融实验：只训练 MLP (Linear Probing)
        model = builder.build_mlp_only_model()
        experiment_id = f"mlp_only_b{args.batch}_e{args.epochs}"

    # 3. 设置评估指标
    metric = evaluate.load("accuracy")
    def compute_metrics(eval_pred):
        logits, labels = eval_pred
        predictions = np.argmax(logits, axis=-1)
        return metric.compute(predictions=predictions, references=labels)

    # 4. 配置训练参数
    # 自动生成不同的输出路径，方便 ipynb 对比
    output_path = os.path.join(Config.SAVE_DIR, experiment_id)

    training_args = TrainingArguments(
        output_dir=output_path,
        learning_rate=args.lr,
        per_device_train_batch_size=args.batch,
        num_train_epochs=args.epochs,
        eval_strategy="epoch",
        save_strategy="epoch",
        logging_steps=10,
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        report_to="none",
        # 针对 MacBook MPS 或普通显存优化的设置
        remove_unused_columns=False
    )

    # 5. 启动 Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_ds,
        eval_dataset=test_ds,
        compute_metrics=compute_metrics,
    )

    print(f"\n🚀 [START] Mode: {args.mode.upper()} | Output: {output_path}")
    trainer.train()

    # 训练结束后的清理或保存操作
    print(f"✅ [FINISHED] Best model saved to {output_path}")

if __name__ == "__main__":
    run_experiment()
```

5.4 inference.py

```python
import torch
import os
from peft import PeftModel, PeftConfig
from transformers import AutoModelForSequenceClassification, AutoTokenize
                                                                        r
class FinancialInferenceEngine:
    def __init__(self, config, checkpoint_path):
        self.config = config
        self.device = torch.device(config.DEVICE)

        # 1. 加载分词器
        self.tokenizer = AutoTokenizer.from_pretrained(config.BASE_MODEL_NAME)

        # 2. 核心机理：自动识别加载模式
        adapter_cfg_path = os.path.join(checkpoint_path, "adapter_config.json")

        if os.path.exists(adapter_cfg_path):
            print(f"🔍 检测到 LoRA 适配器，正在执行 [PEFT] 挂      )
            # 这种方戮蟲樅妥：先读配置，再加载 base，最后挂载
            peft_config = PeftConfig.from_pretrained(checkpoint_path)
            base_model = AutoModelForSequenceClassification.from_pretrained(
                config.BASE_MODEL_NAME,
                num_labels=3
            )
            # 加载并自动处理 modules_to_save (即 classifier)
            self.model = PeftModel.from_pretrained(base_model, checkpoint_path)
        else:
            print(f"🔍 检测到全量/MLP 权重，正在执行 [Direct] 加      )
            # 对于 M蟲;.ΩMly 模式，权重直接就在主模型里，不需要 PeftModel
            self.model = AutoModelForSequenceClassification.from_pretrained(
                checkpoint_path,
                num_labels=3
            )

        self.model.to(self.device)
        self.model.eval()
        print(f"✅ 推理引擎已就   {checkpoint_path}")
                绪:
    def predict_batch(self, test_dataset):
        all_preds = []
        print(f"🚀 正在执行 {len(test_dataset)} 条数据的原生推断...")

        with torch.no_grad():
            for batch in test_dataset:
                # 保持你原版成功的推理逻辑
                inputs =
                    'inp¢t_ids': batch['input_ids'].unsqueeze(0).to(self.device),
                    'attention_mask': batch['attention_mask'].unsqueeze(0).to(self.device),
                    'token_type_ids': batch['token_type_ids'].unsqueeze(0).to(self.device)
                }
                outputs = self.model(**inputs)
                pred_idx = torch.argmax(outputs.logits, dim=-1).item()
                all_preds.append(pred_idx)

        return all_preds
```